PX457 — Assignment 4

Name: Jabir Hussain

Student ID: 2007942

For this assignment, we were provided with skeleton code for a 2D Random Field Ising Model simulation. The skeleton code was structured to allow domain decomposition using MPI, with the primary task being the use of distributed-memory parallelism.

Comms1.c — Using MPI

For the first task, the skeleton code contained serial code for executing the program. In my implementation, I used the MPI calls `MPI_Init(&argc, &argv)`, `MPI_Comm_rank(MPI_COMM_WORLD, &my_rank)` and `MPI_Comm_size(MPI_COMM_WORLD, &p)`. With this, each processor then had access to the global communicator, accessing the unique id (`my_rank`) and the total number of MPI tasks (`p`).

The function `MPI_Wtime()` was used to create timestamps for benchmarking:

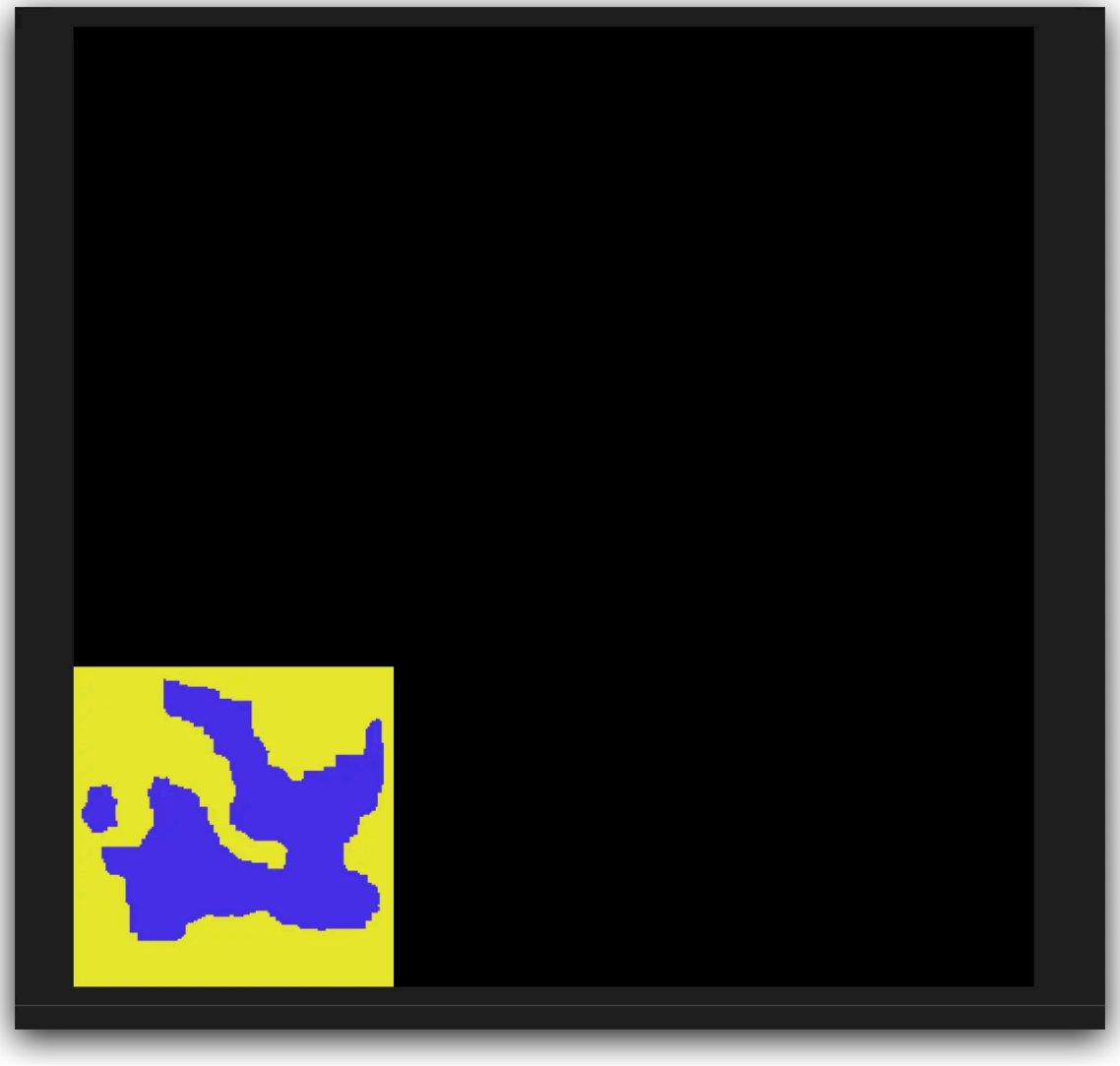
- The timer starts in `comms_initialise`
- The timer ends in `comms_finalise`

`MPI_Finalize()` halts the MPI execution and ensures a clean shutdown.

This implementation has no domain decomposition. The same region of the lattice is always simulated as each processor uses the same global index. In turn, the produced image is only in the bottom left quadrant of the system.

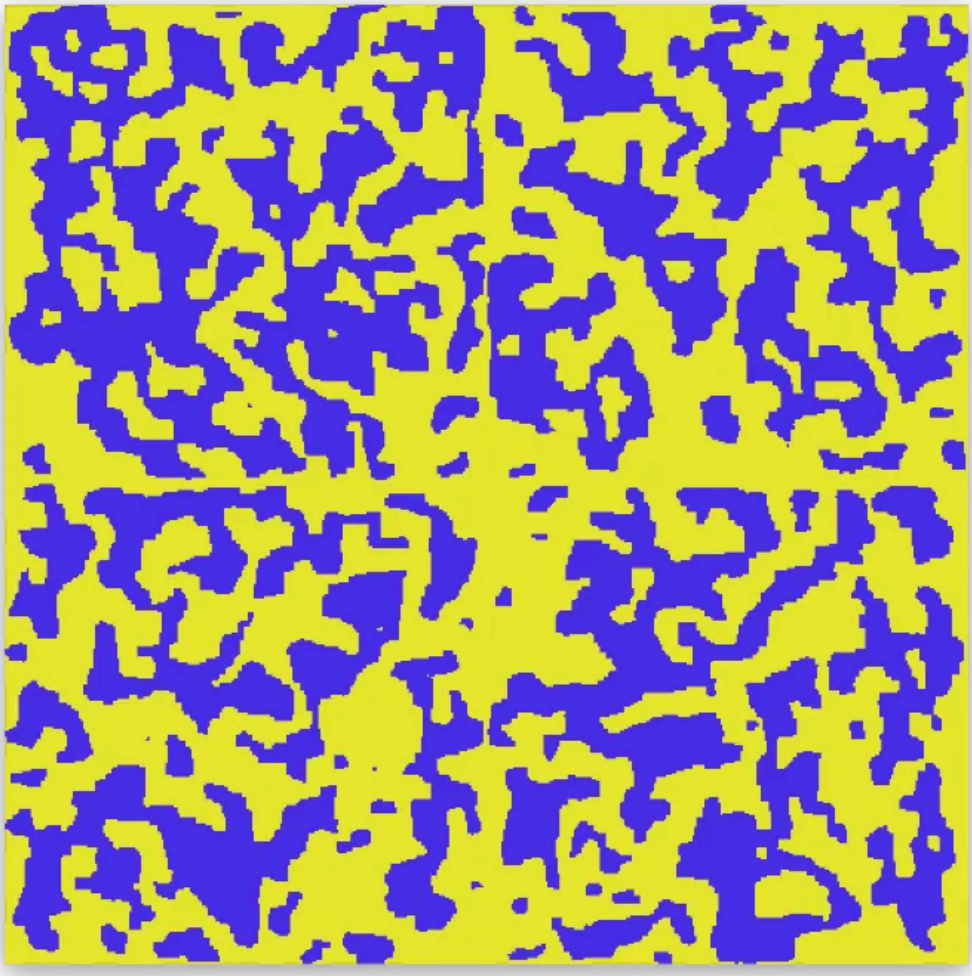
Comms2.c — Cartesian Topology

This task focused on modifying the function `comms_processor_map` to use a 2-dimensional Cartesian communicator topology, created using the MPI call `MPI_Cart_create`. The use of `MPI_Cart_coords` mapped a given linear MPI rank to its Cartesian coordinates on the grid. `MPI_Cart_shift` was used to identify adjacent ranks, converting them into MPI neighbour IDs. A shift along dimension 0 returns horizontal ranks (denoted `left` and `right` in the code), whereas a shift on dimension 1 returns the vertical ranks.



Comms3.c — Processor Data Collection

As shown in the figure above, the output image is still a fraction of the ideal output. The primary task for this exercise was to reconstruct the full $N_{grid} \times N_{grid}$ configuration copying the local `grid_spin` arrays to the `global_grid_spin`. The `comms_get_global_grid` function copies the `grid_spin` block onto `global_grid_spin`. Rank 0 regenerates the lattice using the row-wise spin data and the position data sent from the other ranks. By sending `grid_domain_start` coordinates, rank 0 receives the coordinates from the incoming region and in turn, it calculates the new position on the global grid. The local spin data that is sent to the correct positions in the subdomains, irrespective of MPI rank.

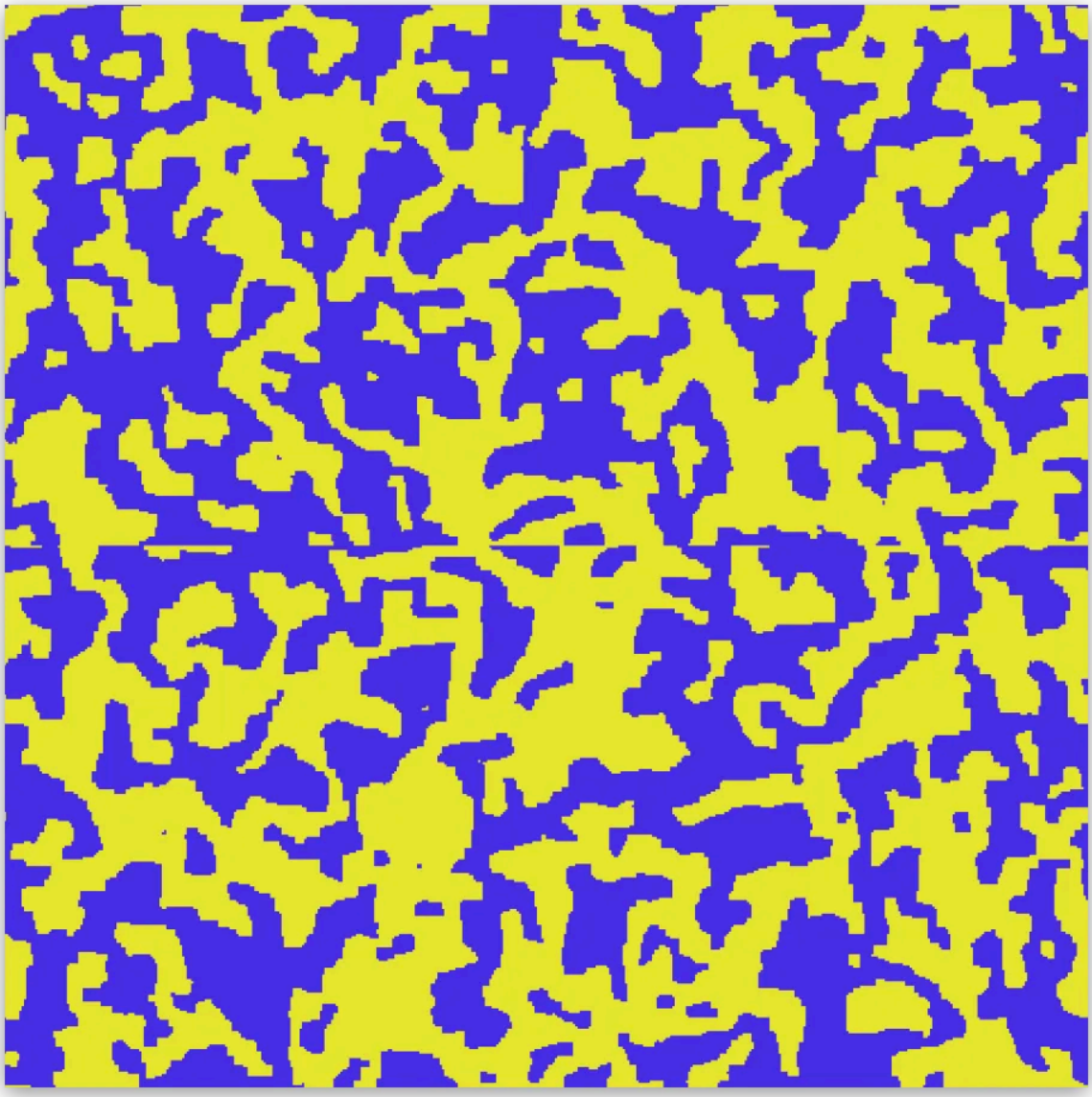


Using 4 processors, the program generates a full 480x480 image across the subdomains, however the boundaries between each subdomain are distinct. This is a result of no halo exchanging between ranks; during the Monte-Carlo iterations, boundary spins use constant halo arrays which are never updated for $p > 1$ as their external neighbours, when they should be using the true spins on corresponding processors.

Comms4.c — Halo Swapping

In this final task, the edge interactions that were overlooked in the previous exercise are to be implemented. More specifically, the `comms_halo_swaps` method is developed so that each rank transmits its boundary spins to the neighbouring subdomains, allocating real variate values for the halo arrays. The MPI call `MPI_Sendrecv` was used for each axis on the grid. Left-right communication sends the left boundary column to the left neighbour and receives the right boundary column from its right neighbour. Conversely, it sends its right boundary column to its right neighbour and receives its left boundary column from its left neighbour. Up-down communication follows the same principle but for the vertical boundaries.

`MPI_Sendrecv` is specifically used for symmetric data exchange as the data is exchanged both ways in a single call, preventing deadlocks. This allows true values to be used in the Monte Carlo energy calculations. In turn, the subdomains can interact across the boundaries, mitigating the artificial uniform grid boundaries as a fully parallel simulation solution.



Timings

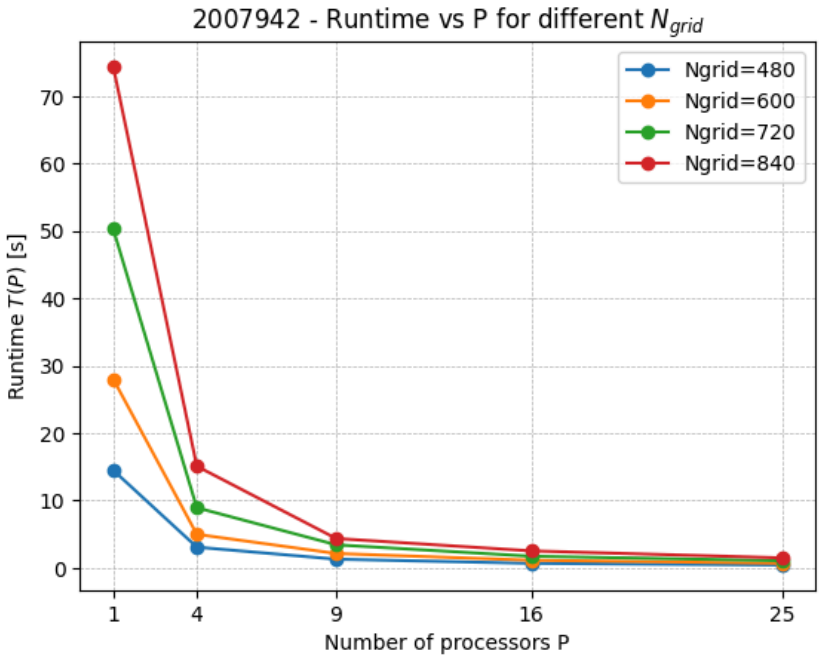
Once the program was fully parallelised with MPI calls, the task was to assess the performance of the code across varying grid side and varying processor count. The speedup ψ was calculated as a function of P with the equation

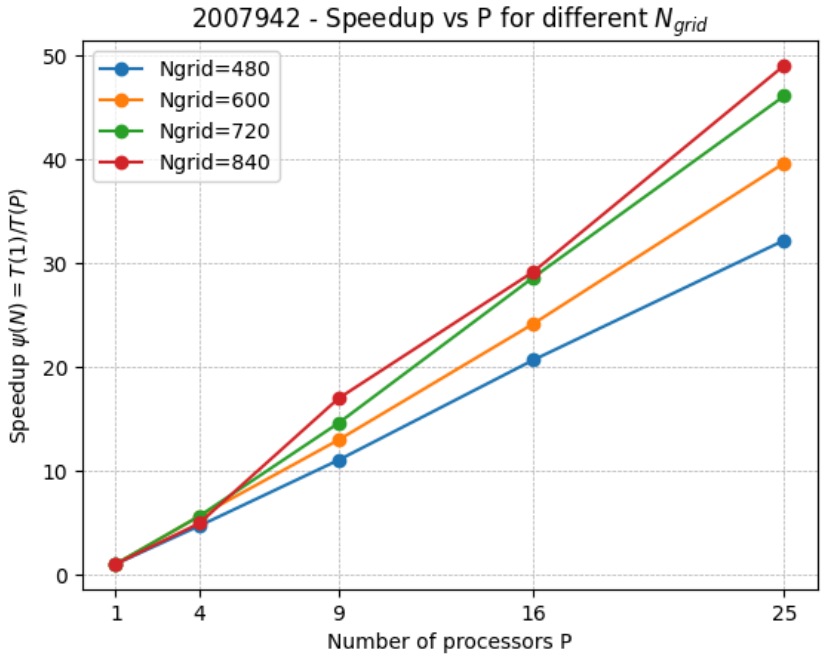
$$\psi(N) = \frac{T(1)}{T(P)}$$

where $T(P)$ is the time taken to run with P MPI tasks, and $T(1)$ is the time taken with 1 MPI task, such that $\psi(1) = 1.0$ exactly. The results were recorded in the table below:

N_grid	P = 1 [s]	P = 4 [s]	P = 9 [s]	P = 16 [s]	P = 25 [s]
480	14.501907	3.102894	1.316276	0.701683	0.450517
600	28.013137	5.002670	2.161382	1.159521	0.706721
720	50.260545	8.946427	3.446620	1.755774	1.090195
840	74.326824	15.097916	4.382737	2.547830	1.515727

These values were plotted to visually highlight the performances.





- For all values of P tested, as N_{grid} grows, so does the runtime. There are more calculations per rank as grid size grows, however the parallelisation shows a significant speedup in calculations as processor count increases.
- For $N_grid = 720$ and 840, the speedup is slightly faster than standard linear scaling. In parallel, each rank only stores a subdomain. Subdomains are more likely to fit in the L2 and L3 caches, reducing memory latency and in turn, executing faster.